

03: Kubernetes Concepts, Deep Dive

Kubernetes interviews for platform roles test whether you understand how the system works, not only how to use `kubectl`. A request enters through the API server, is stored in `etcd`, is placed on a node by the scheduler, and becomes a running container through the `kubelet`. Controllers watch the results and keep the cluster converging toward the desired state.

This chapter follows that path. It then covers networking, storage, the mechanisms that GPU workloads depend on (extended resources, device plugins, and Dynamic Resource Allocation), multi-tenancy, and a set of debugging scenarios. `04-kubernetes-operator.md` and `10-gpu-operator-kubebuilder-deep-dive.md` cover operators and the GPU Operator; this chapter links to them rather than repeating their content.

This chapter covers

- The control plane components and the path of a write request through the API server
- Authentication, authorization, admission, `etcd`, and API Priority and Fairness
- Object metadata, finalizers, garbage collection, and server-side apply
- Workload controllers, QoS classes, sidecars, and autoscaling
- The Pod lifecycle, the scheduling framework, the `kubelet`, and eviction
- Controllers and informers
- Services, `EndpointSlices`, `kube-proxy`, DNS, CNI, and `NetworkPolicy`
- Volumes, configuration, secrets, and quotas
- GPU scheduling with extended resources, device plugins, DRA, and gang scheduling
- Multi-tenancy and GPU cloud control planes
- Debugging scenarios and interview questions

Note: This chapter describes Kubernetes v1.37, released on 2026-08-26. Feature states are given with the release in which they changed. The upstream documentation's main branch describes the next release, so some features in section 8.7, such as the Workload and PodGroup scheduling APIs, are beta and may not be enabled on a given cluster.

Contents

1. Cluster architecture and the API server	5
1.1 Components	5
1.2 The path of a write request	5
1.3 Authentication	6
1.4 Authorization	7

1.5 Admission.....	7
1.6 etcd and watches	8
1.7 API Priority and Fairness	8
1.8 Extending the API	9
2. Objects and metadata	10
2.1 Object structure.....	10
2.2 Labels, selectors, and owner references	10
2.3 Namespaces.....	10
2.4 Finalizers and deletion	11
2.5 Garbage collection.....	11
2.6 Useful API features.....	11
3. Workload resources.....	13
3.1 Pods	13
3.2 Requests, limits, and QoS classes.....	13
3.3 Sidecar containers	14
3.4 ReplicaSets and Deployments	14
3.5 StatefulSets	15
3.6 DaemonSets	15
3.7 Jobs and CronJobs.....	15
3.8 Autoscaling	16
3.9 PodDisruptionBudgets	16
4. The Pod lifecycle, scheduling, and eviction	18
4.1 From kubectl apply to a running container	18
4.2 Pod phase, container state, and conditions	18
4.3 The scheduling framework	19
4.4 Controlling placement	19
4.5 The kubelet	20
4.6 Probes and restarts.....	21
4.7 Termination.....	21
4.8 Node heartbeats and failure	22
4.9 Node-pressure eviction.....	22
5. Controllers and informers	23
5.1 The controller pattern	23
5.2 client-go internals	23
5.3 controller-runtime terms	23
5.4 Avoiding conflicting writes	24
6. Services and cluster networking.....	25
6.1 Service types.....	25
6.2 EndpointSlices	25
6.3 How Service traffic is forwarded	26
6.4 DNS.....	26
6.5 CNI and Pod networking.....	27

6.6 NetworkPolicy	27
7. Storage, configuration, and secrets	28
7.1 PersistentVolumes, claims, and StorageClasses	28
7.2 ConfigMaps and Secrets	28
7.3 ResourceQuota and LimitRange	29
8. Scheduling GPU workloads	30
8.1 Extended resources	30
8.2 How a device plugin works	30
8.3 GPU resources and labels on a node	31
8.4 MIG, time-slicing, and MPS	32
8.5 NUMA alignment	33
8.6 Dynamic Resource Allocation (DRA)	33
8.7 Gang scheduling	35
8.8 Requirements for a working GPU Pod	35
9. Multi-tenancy and GPU cloud control planes	36
9.1 Soft and hard multi-tenancy	36
9.2 Controls by layer	36
9.3 A GPU cloud control plane built on Kubernetes	36
10. kubectl, client-go, and apply	38
10.1 kubectl commands	38
10.2 Calling the API directly	38
10.3 client-go and controller-runtime	39
10.4 Create, replace, apply, and patch	39
11. Debugging scenarios	40
11.1 A Pod stuck in Pending	40
11.2 A Pod stuck in ContainerCreating	40
11.3 A container in CrashLoopBackOff	41
11.4 A node in NotReady	41
11.5 A Service with endpoints that does not respond	41
11.6 DNS failures	42
11.7 A GPU Pod cannot use the GPU	42
11.8 Controller and API server problems	42
12. Interview questions	44
What happens between kubectl apply and a running Pod?	44
What happens when a node fails while running Pods?	44
Why is a Deployment rollout stuck?	44
How does a Service route only to healthy Pods?	44
How are GPU requests scheduled?	45
Should MIG be exposed through the device plugin or DRA?	45
Why does distributed training need gang scheduling?	45
What is the difference between requests and limits?	45
What happens if etcd is slow or lost?	45

How would you isolate two teams on one GPU cluster?.....	46
A controller is reconciling constantly and the API server is slow. What do you check?	46
Summary	47
Further reading	48

1. Cluster architecture and the API server

1.1 Components

Kubernetes consists of control loops that coordinate through a single API. Each component has a defined responsibility:

Component	Responsible for	Communicates with
kube-apiserver	Serving the REST API; the only component that reads and writes etcd	etcd and all clients
etcd	Durable storage of all cluster state	The API server only
kube-scheduler	Choosing a node for each unscheduled Pod	The API server
kube-controller-manager	Built-in controllers: Deployment, ReplicaSet, Job, node lifecycle, EndpointSlice, garbage collection, and others	The API server
cloud-controller-manager	Cloud-specific controllers: load balancers, routes, and node lifecycle	The API server and cloud APIs
kubelet	Running the Pods on its node: sandboxes, containers, volumes, probes, and status	The API server, the container runtime (CRI), CSI drivers, and device plugins
kube-proxy (optional)	Programming Service forwarding rules on its node	The API server
Container runtime (containerd, CRI-O)	Running containers	The kubelet, over CRI

Add-ons such as CoreDNS, the CNI plugin, the NVIDIA device plugin, and the GPU Operator are ordinary workloads that run in kube-system or their own namespaces.

Two design principles follow from this architecture:

1. **Components communicate through API objects.** A controller does not act on a node directly. It writes an object, and another component reacts to it. The GPU Operator, for example, does not load the nvidia kernel module itself; it creates a driver DaemonSet, and the kubelet on each node runs the Pods that load it.
2. **Controllers are level-triggered.** A controller compares desired state (the spec) with observed state and acts until they match. It does not need to receive every event, so a controller that restarts reads the current state and continues correctly.

1.2 The path of a write request

Every change to the cluster, whether `kubectl apply`, a controller updating status, or a kubelet reporting node status, passes through the same stages in the API server:

```

client (kubectl / controller / kubelet)
|  HTTPS: JSON, or protobuf for performance
▼
TLS termination + authentication      (identity of the caller)
▼
authorization (RBAC / node / webhook) (whether the verb is permitted here)
▼
mutating admission                    (built-ins + MutatingAdmissionWebhooks)
▼
schema validation / defaulting        (types, required fields, defaults)
▼
validating admission                  (built-ins + ValidatingAdmissionWebhooks)
▼
conversion to the storage version
▼
persist to etcd (with optimistic concurrency)
▼
watch event fan-out to all watchers

```

Mutating admission runs before validation, because mutation is where defaults and injected sidecars are added, and the final object must be validated. If a mutating webhook could change an object after validation, it could create objects that validation would have rejected.

Webhooks can be limited to specific requests with match conditions (GA in v1.30), and each has a failure policy. With `Fail`, the default, the write is rejected if the webhook cannot be reached; with `Ignore`, the write proceeds. A webhook with `failurePolicy: Fail` that becomes unavailable blocks every write it matches, and it is a common cause of cluster-wide outages.

Because the API server is the only component that writes to etcd, authorization, admission, and auditing are all enforced in one place.

1.3 Authentication

The API server supports several authentication methods, including:

- X.509 client certificates, used by kubelets, controller components, and administrator kubeconfig files
- Bearer tokens
- OpenID Connect, for human users and their groups
- Webhook token review
- ServiceAccount tokens, for workloads

Each Pod runs as a ServiceAccount. Since v1.24, the kubelet obtains a short-lived, audience-bound token for it through the `TokenRequest` API and mounts it in a projected volume, instead of mounting a long-lived token stored in a `Secret`. The token specifies its audience and expiration, and the kubelet refreshes it before it expires. Long-running Pods therefore never hold a credential valid for years. For Pods that do not call the API, set `automountServiceAccountToken: false` to remove the credential entirely.

The Node authorizer limits each kubelet to objects related to its own node, and the NodeRestriction admission plugin prevents a kubelet from modifying other nodes or changing protected labels on its own node.

1.4 Authorization

RBAC is the standard authorization mode:

Object	Scope
Role and RoleBinding	One namespace
ClusterRole and ClusterRoleBinding	The whole cluster
RoleBinding that references a ClusterRole	The ClusterRole's permissions, within one namespace

The verbs are get, list, watch, create, update, patch, delete, and deletecollection. Three details cause frequent mistakes:

- **list and watch are separate from get.** A controller with only get permission cannot fill its informer cache, which lists and watches.
- **RBAC has no deny rules.** Permissions only add up. To restrict access, do not grant it, or enforce restrictions with admission policies.
- **The system:masters group bypasses RBAC completely.** Do not grant membership in it.

Operators should request only the permissions they use. Kubebuilder generates RBAC rules from markers:

```
// +kubebuilder:rbac:groups=apps,resources=deployments,verbs=get;list;watch;create;update;patch;delete
// +kubebuilder:rbac:groups=apps,resources=deployments/status,verbs=get;update;patch
```

1.5 Admission

Admission enforces policy on objects before they are stored:

- **Built-in admission plugins** include NamespaceLifecycle, LimitRanger (applies LimitRange defaults and limits), ResourceQuota, ServiceAccount, NodeRestriction, TaintNodesByCondition, Priority, StorageObjectInUseProtection, and Pod Security Admission (the baseline and restricted profiles).
- **Mutating webhooks** add defaults, sidecar containers, labels, and annotations. They can be called more than once for the same request, so they must be idempotent, and they must respond quickly.
- **Validating webhooks** reject objects that violate rules.

For simple rules, avoid webhooks. CEL validation rules in a CRD schema (x-kubernetes-validations) and ValidatingAdmissionPolicy objects run inside the API server and cannot become unavailable. Use a webhook for checks that depend on other objects or on data

outside the request. A validating webhook receives the object after mutation, so defaults added by mutating admission are already present.

1.6 etcd and watches

etcd is a key-value store replicated with the Raft consensus algorithm. A leader orders all writes, and a write is committed when a majority of members (a *quorum*) has recorded it. This has several operational consequences:

- **Cluster size.** A three-member cluster tolerates one failed member, and a five-member cluster tolerates two. A two-member cluster tolerates none, because both members are needed for a majority, so it is no more available than one member. Use an odd number of members.
- **Disk latency.** etcd calls `fsync` for every write, so disk latency limits the API server's write throughput. Run etcd on fast local SSDs. A slow etcd appears as slow `kubectl apply`, slow controller convergence, and repeated leader elections in controllers.
- **Maintenance.** etcd keeps old revisions until they are compacted, and its database file must be defragmented to reclaim space. Schedule both.
- **Encryption at rest.** The API server encrypts resources such as Secrets before storing them, preferably with the KMS v2 provider. This protects the data on etcd's disk. It does not restrict API access; RBAC still controls who can read Secrets.

The API server maintains a **watch cache** for each resource type, built from etcd watches. It serves most list requests without reading etcd, and it lets clients start a watch from a recent `resourceVersion`. A `resourceVersion` is an opaque marker; do not compare or compute with it.

Rules for `resourceVersion`:

- **Optimistic concurrency.** An update includes the `resourceVersion` the client read. If another client wrote the object in the meantime, the update fails with **409 Conflict**. The client must read the object again and reapply its change. Retrying with the old object would overwrite the other client's change.
- **Expired watches.** A watch that starts from a `resourceVersion` older than the cache's history fails with **410 Gone**. The client must list again and watch from the new `resourceVersion`. The client-go Reflector does this automatically.
- **Pagination and bookmarks.** Lists are paginated with `limit` and `continue`. Watches can request bookmark events, which update the client's `resourceVersion` without sending objects, so a later reconnect does not start from an expired version.

1.7 API Priority and Fairness

API Priority and Fairness (APF, stable in v1.29) limits how many requests the API server processes concurrently, and it divides that capacity among classes of clients. Each request is matched by a `FlowSchema` to a `PriorityLevelConfiguration`, and each priority level has its own share of concurrency. The default configuration defines separate levels for leader election, node requests, built-in controllers, and other traffic.

As a result, a misbehaving client, such as a controller that lists all objects in a tight loop, cannot prevent other clients from using the API. Its requests are queued and then rejected with 429 Too Many Requests, and its latency increases, while other priority levels continue to be served.

1.8 Extending the API

Kubernetes can be extended with new resource types in two ways:

- **CustomResourceDefinitions (CRDs).** You define a resource type, the API server stores and serves it, and you write a controller for it. Operators use this approach.
- **The aggregation layer.** You register a separate API server that handles a group of resources itself, including storage. `metrics-server` and custom metrics adapters use this approach.

API versions apply to whole resources, not to individual fields. The API server can serve several versions of a resource and convert between them and a single storage version, so during a migration the same object can be read as `v1beta1` or `v1`. Beta APIs can still change; `v1` APIs come with a compatibility commitment.

2. Objects and metadata

2.1 Object structure

Every object has `apiVersion`, `kind`, and `metadata`, and nearly every object has `spec` (the desired state) and `status` (the observed state). These fields carry operational meaning:

Field	Meaning
<code>metadata.name</code> , <code>metadata.namespace</code>	Identity. A name is unique for its resource type within a namespace
<code>metadata.labels</code>	Key-value pairs that selectors can query
<code>metadata.annotations</code>	Key-value metadata that cannot be selected on
<code>metadata.ownerReferences</code>	Links from a dependent object to its owners, used by garbage collection
<code>metadata.finalizers</code>	Keys that must be removed before the object can be deleted
<code>metadata.generation</code>	Incremented when the spec changes
<code>status.observedGeneration</code>	The generation that the controller last processed
<code>metadata.managedFields</code>	Which field manager owns each field, for server-side apply
<code>status.conditions</code>	Observations with a type, a True, False, or Unknown status, a reason, and a message

If `status.observedGeneration` is lower than `metadata.generation`, the controller has not yet processed the latest spec change, and the status describes an older spec.

2.2 Labels, selectors, and owner references

Labels decouple objects from each other. A `ReplicaSet` finds its Pods with its `spec.selector`, and a `Service` finds its endpoints in the same way. Two rules follow:

- **Workload selectors are immutable.** The selector of a `Deployment`, `StatefulSet`, or `DaemonSet` cannot be changed in `apps/v1`. A changed selector could release Pods the controller manages or adopt Pods it does not, so the API server rejects the change.
- **Owner references, not labels, define ownership.** Labels are a flat index; `ownerReferences` form a graph that garbage collection follows. A controller should delete only objects it owns, not every object that matches a label.

2.3 Namespaces

A namespace scopes names and is the unit for RBAC bindings and resource quotas. It is not an isolation mechanism. Without `NetworkPolicy`, Pods in different namespaces can communicate freely, and without admission controls, a privileged Pod in any namespace

can access its host. A namespace for each tenant is the starting point of a tenancy model, not the complete model (section 9).

2.4 Finalizers and deletion

A finalizer is a string in `metadata.finalizers`. When a client deletes an object that has finalizers, the API server sets `metadata.deletionTimestamp` and keeps the object. The controller responsible for each finalizer performs its cleanup and removes its finalizer. When the list is empty, the API server deletes the object.

Problems with finalizers are common:

- **Objects stuck in Terminating.** If the controller that owns a finalizer is not running, or fails before removing it, the object is never deleted.
- **Repeated cleanup.** A controller can restart after cleanup but before removing the finalizer, so cleanup must be idempotent.
- **Namespaces stuck in Terminating.** Namespace deletion waits for every object in the namespace to be deleted. A namespace that does not finish deleting usually contains an object with an unresolved finalizer, or has resources from an API service that is unavailable.

2.5 Garbage collection

The garbage collector deletes dependent objects whose owners no longer exist. A namespaced dependent can have owners only in its own namespace, or cluster-scoped owners. There are three deletion modes:

Mode	Behavior
Background (default)	The owner is deleted immediately, and the garbage collector deletes dependents afterward
Foreground	The owner receives a <code>deletionTimestamp</code> and the <code>foregroundDeletion</code> finalizer, and remains until dependents with <code>blockOwnerDeletion: true</code> are deleted
Orphan	The owner is deleted, and the dependents remain with their owner references removed

Choose the mode with `kubectl delete --cascade=background|foreground|orphan`.

2.6 Useful API features

- **Discovery.** `GET /api`, `GET /apis`, and `kubectl api-resources` list the available groups, versions, kinds, and resource names.
- **Schema inspection.** `kubectl explain pod.spec.containers --recursive` reads the OpenAPI schema that the API server publishes.
- **Field selectors** filter on the server by supported fields such as `metadata.name`, `metadata.namespace`, and `status.phase`. They are different from label selectors and much cheaper than filtering on the client.

- **Watches** are long-running GET requests with `?watch=1` that stream changes. Controllers use them instead of polling.
- **Dry run** (`?dryRun=All`) runs the complete admission and validation path without storing anything. Use it to test webhooks and manifests.
- **Server-side apply** records field ownership in `managedFields`. When two managers set the same field to different values, apply reports a conflict instead of silently overwriting, and `--force-conflicts` takes ownership explicitly.

3. Workload resources

3.1 Pods

The Pod is the smallest unit that Kubernetes schedules and runs:

- **Shared network.** Containers in a Pod share one network namespace, and therefore one IP address and `localhost`. They can also share volumes.
- **The sandbox.** The kubelet starts a Pod by asking the container runtime to create a sandbox. On Linux, the sandbox is a pause container that holds the network and IPC namespaces; the application containers join them. The Pod's IP address therefore exists before any application container starts. The `PodReadyToStartContainers` condition (stable in v1.37, formerly `PodHasNetwork`) reports that the sandbox and network are ready.
- **Shared process namespace.** With `shareProcessNamespace: true`, containers in the Pod can see each other's processes. This helps with debugging, but it changes signal handling and the contents of `/proc`, and the pause process becomes PID 1.
- **No survival across node loss.** A Pod is bound to its node. If the node fails, the Pod is not moved; a workload controller creates a replacement.

3.2 Requests, limits, and QoS classes

The combination of resource requests and limits in a Pod determines its quality of service (QoS) class:

QoS class	Condition	Effect
Guaranteed	Every container has CPU and memory requests equal to its limits, and all are set	Evicted last; eligible for exclusive CPUs with the static CPU Manager policy
Burstable	At least one container has a CPU or memory request or limit, but the Pod is not Guaranteed	Receives its requests and can use more, up to its limits or the node's capacity
BestEffort	No container has any CPU or memory request or limit	Evicted first under node pressure

Rules to remember:

- **Requests are for scheduling; limits are enforced by the kernel.** A container that exceeds its memory limit is killed by the cgroup OOM killer. A container that exceeds its CPU limit is throttled, not killed.
- **Init containers are counted differently.** The Pod's effective request is the larger of two values: the sum of the requests of the application and sidecar containers, or the largest request of any init container. Pod overhead is added to the result. A

large init container can therefore reserve resources that the running Pod never uses.

- **QoS class does not change after creation.** An in-place resize that would change the class is rejected.
- **Memory QoS** (beta in v1.37) is an opt-in kubelet feature that uses cgroup v2 `memory.high` and memory protection settings.

3.3 Sidecar containers

A native sidecar is an init container with `restartPolicy: Always` (the `SidecarContainers` feature: beta in v1.29, stable in v1.33). Native sidecars solve three problems that sidecars implemented as regular containers had:

- **Start and stop order.** Sidecars start before the application containers and stop after them, in reverse order. A log shipper or proxy keeps running while the application finishes its work.
- **Job completion.** A sidecar does not keep a Job running after its main container exits.
- **Independent restarts.** A sidecar restarts on its own if it fails.

For example, to run an Envoy proxy beside a Triton Inference Server and ensure that the proxy is ready first, declare the proxy as a native sidecar with a startup probe. Before native sidecars, this required workarounds with `preStop` hooks and scripts.

3.4 ReplicaSets and Deployments

A `ReplicaSet` keeps a specified number of Pods that match its selector, creating or deleting Pods as needed. A `Deployment` manages `ReplicaSets` and adds rollout behavior:

- **Rollouts.** Changing `spec.template` creates a new `ReplicaSet` and gradually moves Pods to it. Changing only the replica count scales the current `ReplicaSet` without a rollout.
- **Rollout speed.** `strategy.rollingUpdate.maxSurge` (default 25%) limits how many extra Pods can exist during a rollout, and `maxUnavailable` (default 25%) limits how many desired Pods can be unavailable.
- **History.** `revisionHistoryLimit` sets how many old `ReplicaSets` are kept for rollback.
- **Failure detection.** If a rollout makes no progress for `progressDeadlineSeconds` (default 600), the `Deployment` reports it as failed. It does not roll back automatically.
- **Commands.** `kubectl rollout status, history, undo, pause, and resume`.

A rollback scales the previous `ReplicaSet` up and the current one down. It does not restore application data, so database schema changes must be reversible on their own.

3.5 StatefulSets

A StatefulSet gives each Pod a stable identity, which databases and distributed training jobs need:

- **Stable names.** Pods are named `<statefulset-name>-<ordinal>`, such as `web-0` and `web-1`. `.spec.ordinals.start` changes the first ordinal, and `.spec.serviceName` names the headless Service that gives each Pod a DNS name.
- **Ordering.** With `podManagementPolicy: OrderedReady` (default), Pods are created one at a time in order and deleted in reverse order. With `Parallel`, they are created and deleted together.
- **Stable storage.** Each Pod can have its own `PersistentVolumeClaim` from `volumeClaimTemplates`. `persistentVolumeClaimRetentionPolicy` controls whether the claims are deleted when the StatefulSet is deleted or scaled down.
- **Updates.** `RollingUpdate` (default) replaces Pods from the highest ordinal down. `partition` limits the update to ordinals at or above a value, for canary releases. `maxUnavailable` allows more than one Pod to be updated at a time; it was alpha from v1.24 and is beta and enabled by default from v1.35, but it is not GA in v1.37, so confirm it is enabled before depending on it.
- **Ordinal label.** The `apps.kubernetes.io/pod-index` label holds each Pod's ordinal.

Distributed training needs stable ranks and network identities. A StatefulSet that derives the rank from its hostname still works, but current releases are moving toward the Workload and PodGroup APIs with gang scheduling (section 8.7).

3.6 DaemonSets

A DaemonSet runs one Pod on each eligible node. It is the standard way to run node agents: GPU driver containers, the NVIDIA Container Toolkit, the device plugin, the DCGM exporter, CNI plugins, node-exporter, and log collectors.

Two details are important on GPU nodes:

- **DaemonSet Pods keep running on unhealthy nodes.** The DaemonSet controller pins each Pod to its node with node affinity on `metadata.name`. The node lifecycle controller adds taints such as `node.kubernetes.io/not-ready`, `unreachable`, `disk-pressure`, `memory-pressure`, `pid-pressure`, and `unschedulable` to unhealthy nodes, and the DaemonSet controller adds matching tolerations to its Pods automatically. Monitoring agents therefore keep running on the nodes where they are most needed.
- **Update strategy.** `RollingUpdate` (default) supports `maxUnavailable` (default 1) and `maxSurge` (default 0; GA since v1.25). `OnDelete` replaces a Pod only when someone deletes it, which suits drivers that must not restart until the node has been drained.

3.7 Jobs and CronJobs

A Job runs Pods until a specified number complete successfully:

Field	Purpose
<code>completions</code>	The number of successful Pods required
<code>parallelism</code>	The number of Pods that can run at once
<code>completionMode</code>	NonIndexed (default) or Indexed, which gives each Pod a stable index for sharding work
<code>backoffLimit</code>	The number of retries before the Job fails
<code>activeDeadlineSeconds</code>	The maximum run time of the Job
<code>podFailurePolicy</code>	Rules that mark some failures as fatal, such as a specific exit code for invalid input, and others as retryable
<code>ttlSecondsAfterFinished</code>	Deletes the finished Job and its Pods after a delay
<code>successPolicy</code> , <code>podReplacementPolicy</code>	When an indexed Job counts as successful, and when failed Pods are replaced

A CronJob creates Jobs on a schedule and adds:

- `concurrencyPolicy`: Allow, Forbid, or Replace. Forbid is the safe choice for GPU jobs that would compete for the same devices.
- `startingDeadlineSeconds`: a run that cannot start within this time is skipped rather than started late.
- `timeZone`: the time zone in which the schedule is interpreted.
- `successfulJobsHistoryLimit` and `failedJobsHistoryLimit`.

If the CronJob controller finds more than 100 missed schedules since the last run, it stops scheduling that CronJob and logs an error. Setting `startingDeadlineSeconds` limits how far back the controller counts.

3.8 Autoscaling

- **The Horizontal Pod Autoscaler (HPA)** changes the replica count based on metrics: CPU, memory, or custom and external metrics. Resource metrics come from `metrics-server`.
- **The Vertical Pod Autoscaler (VPA)** recommends or sets CPU and memory requests. It does not manage extended resources such as GPUs, which are integers, cannot be overcommitted, and usually require replacing the Pod to change.
- **Node autoscalers** (Cluster Autoscaler, Karpenter, managed node pools) add and remove nodes. They run outside the core control plane and act on a cloud provider's API.

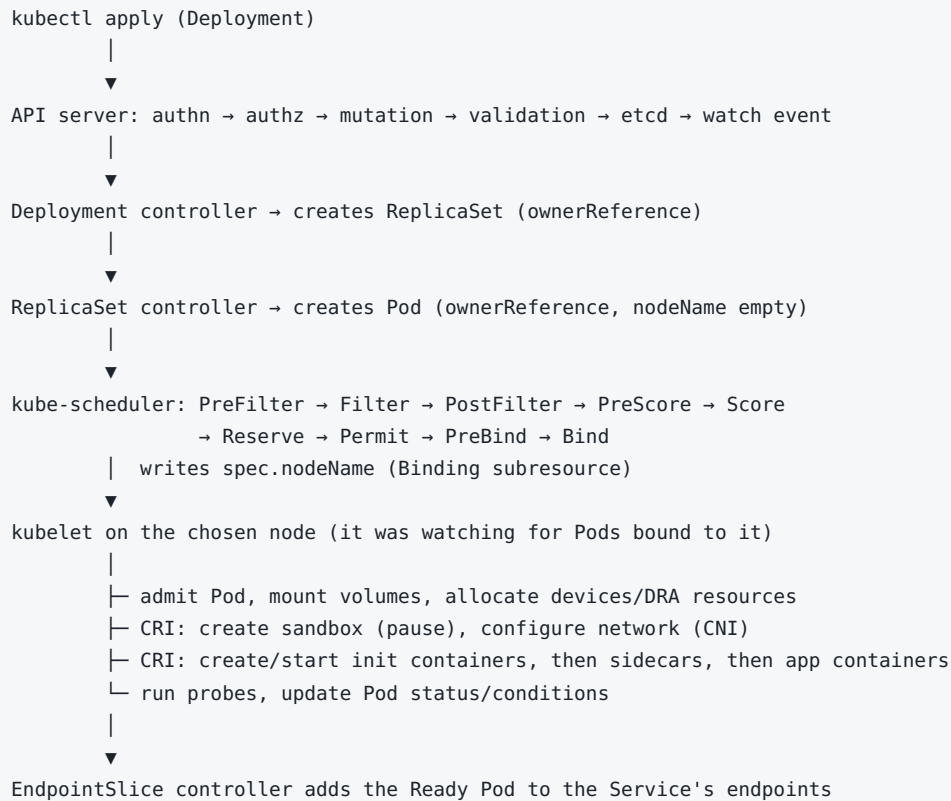
3.9 PodDisruptionBudgets

A PodDisruptionBudget (PDB) limits voluntary disruptions made through the **Eviction API**; it cannot prevent a node from failing. `minAvailable` or `maxUnavailable` defines the budget. `unhealthyPodEvictionPolicy` (IfHealthyBudget or AlwaysAllow) controls whether

Pods that are not ready can be evicted even when the budget is exhausted. `kubectl drain` evicts Pods through the Eviction API, so a PDB prevents a node maintenance operation from taking too many replicas of a quorum-based service down at once.

4. The Pod lifecycle, scheduling, and eviction

4.1 From `kubectl apply` to a running container



Two properties of this sequence are worth stating in an interview. The scheduler only writes `spec.nodeName` through the Binding subresource; it never starts containers. And each component observes the previous step through its own watch, so the whole sequence is asynchronous and eventually consistent.

4.2 Pod phase, container state, and conditions

`status.phase` is a coarse summary: Pending, Running, Succeeded, or Failed, or Unknown when the node cannot be reached. `CrashLoopBackOff` and `Terminating` are not phases. `kubectl` displays them based on container states and the deletion timestamp. Each container has its own state, Waiting, Running, or Terminated, with a reason and, for terminated containers, an exit code.

Pod conditions give more detail: `PodScheduled`, `PodReadyToStartContainers`, `Initialized`, `ContainersReady`, `Ready`, `DisruptionTarget`, `PodResizePending`, and `PodResizeInProgress`.

A Pod is Ready when all its containers are ready and every condition listed in its `readinessGates` is True. Readiness gates let an external controller, such as a load balancer controller, add its own requirement for readiness.

4.3 The scheduling framework

The scheduler processes each Pod in a **scheduling cycle**, which selects a node, followed by a **binding cycle**, which applies the decision. Scheduling cycles run one Pod at a time; binding cycles can run concurrently. Plugins implement the following extension points:

Extension point	Purpose
PreEnqueue	Decide whether a Pod can enter the active queue; used by scheduling gates and gang scheduling
QueueSort	Order the Pods in the queue, usually by priority
PreFilter	Compute state for the cycle, or reject the Pod early
Filter	Exclude nodes that cannot run the Pod; evaluated for nodes in parallel
PostFilter	Runs only when no node passed Filter; preemption is implemented here
PreScore, Score, NormalizeScore	Rank the feasible nodes
Reserve, Unreserve	Reserve resources on the chosen node before binding, and release them if a later step fails
Permit	Approve, deny, or delay binding; gang scheduling holds Pods here
PreBind, Bind	Prepare resources such as volumes, then write the binding
PostBind	Informational cleanup after a successful bind

Details to know:

- **Reservation avoids races.** Resources are reserved before the asynchronous bind, so the next scheduling cycle does not assign them again. If a later step fails, Unreserve runs in reverse order, and it must not fail.
- **Queueing hints** (stable in v1.34) let the scheduler retry an unschedulable Pod only when an event that could make it schedulable occurs, instead of after every change in the cluster.
- **Multiple schedulers.** A cluster can run several scheduler profiles, or a second scheduler. A Pod chooses one with `spec.schedulerName`.

4.4 Controlling placement

From the simplest mechanism to the most expressive:

Mechanism	Behavior
<code>spec.nodeName</code>	Assigns the Pod to a node directly, bypassing the scheduler
<code>nodeSelector</code>	Requires node labels with exact values
Node affinity	Required (<code>requiredDuringSchedulingIgnoredDuringExecution</code>) or weighted preferred (<code>preferredDuringSchedulingIgnoredDuringExecution</code>) rules, with the operators <code>In</code> , <code>NotIn</code> , <code>Exists</code> , <code>DoesNotExist</code> , <code>Gt</code> , and <code>Lt</code>
Inter-Pod affinity and anti-affinity	Places Pods near or away from other Pods within a <code>topologyKey</code> , such as a node or zone. Expensive to evaluate in large clusters
Topology spread constraints	Spreads Pods evenly across domains with <code>maxSkew</code> , <code>topologyKey</code> , <code>whenUnsatisfiable</code> (<code>DoNotSchedule</code> or <code>ScheduleAnyway</code>), <code>minDomains</code> , <code>nodeAffinityPolicy</code> , and <code>nodeTaintsPolicy</code> . Prefer them to anti-affinity for spreading replicas across zones
Taints and tolerations	A taint repels Pods that do not tolerate it. <code>NoSchedule</code> blocks new Pods, <code>PreferNoSchedule</code> avoids placing them, and <code>NoExecute</code> also evicts running Pods. <code>tolerationSeconds</code> lets a Pod stay for a limited time after a <code>NoExecute</code> taint is added
Priority and preemption	A <code>PriorityClass</code> (cluster-scoped; user-defined values up to 1,000,000,000) allows the scheduler to evict lower-priority Pods when no node fits. <code>preemptionPolicy</code> : Never gives a Pod high queue priority without preempting others. The built-in classes <code>system-cluster-critical</code> (2,000,000,000) and <code>system-node-critical</code> (2,000,001,000) protect essential add-ons

`kubectl cordon` marks a node unschedulable. `kubectl drain` cordons the node and evicts its Pods through the Eviction API, respecting `PodDisruptionBudgets`.

4.5 The kubelet

The kubelet reconciles the Pods assigned to its node. It:

- Watches for Pods bound to its node, and admits and starts them
- Calls the container runtime through CRI to create sandboxes and containers
- Mounts volumes, calling CSI drivers
- Allocates devices through the device manager and calls DRA drivers' `NodePrepareResources`
- Runs startup, liveness, and readiness probes
- Reports Pod status and node status, and renews the node's Lease as a heartbeat
- Removes unused images and exited containers
- Evicts Pods when node resources run low

The kubelet detects container state changes with the Pod Lifecycle Event Generator (PLEG), which by default polls the runtime; `EventedPLEG` uses runtime events instead. When the runtime responds slowly, the kubelet reports PLEG is not healthy, and the node can alternate between `Ready` and `NotReady`.

4.6 Probes and restarts

Probe	Effect of failure
startupProbe	Other probes are disabled until it succeeds. If it does not succeed in time, the container is restarted
livenessProbe	The container is restarted
readinessProbe	The Pod is removed from Service endpoints; nothing is restarted

Probes can use `exec`, `httpGet`, `tcpSocket`, or `grpc`. gRPC probes suit inference servers such as Triton that expose the gRPC health checking protocol.

Restart behavior:

- The Pod's `restartPolicy` is `Always` (default), `OnFailure`, or `Never`. Individual containers can override it with restart rules (`ContainerRestartRules`, beta in v1.35).
- After repeated failures, the kubelet waits longer before each restart. `kubectl` shows this waiting state as `CrashLoopBackOff`. `KubeletCrashLoopBackOffMax` (beta in v1.35) makes the maximum delay configurable.
- A container that runs successfully for long enough resets the delay.
- Sidecar containers are always restarted, regardless of the Pod's restart policy.

4.7 Termination

When a Pod is deleted, termination proceeds as follows:

1. The API server sets `metadata.deletionTimestamp` and the grace period (`terminationGracePeriodSeconds`, default 30 seconds). The Pod remains visible, and `kubectl` shows it as `Terminating`.
2. The `EndpointSlice` controller marks the Pod's endpoints `terminating: true` and `ready: false`, while serving continues to reflect the Pod's readiness. Load balancers that understand these conditions can drain existing connections instead of cutting them.
3. The kubelet runs each container's `preStop` hook, if one is defined. If the hook is still running when the grace period ends, the kubelet extends the period once by about two seconds.
4. The kubelet has the runtime send the stop signal to each container's PID 1. The signal is the image's `STOPSIGNAL` if set, which the `ContainerStopSignals` feature lets a container override, and `SIGTERM` otherwise.
5. When the grace period expires, the runtime sends `SIGKILL` to any remaining processes. The Pod reaches a terminal phase, and the API object is removed.

With native sidecars, the kubelet stops the sidecars only after all application containers have stopped, and stops them in reverse order of their definition.

On GPU nodes, a CUDA process should handle `SIGTERM` and release its GPU context. If it does not exit, it is killed with `SIGKILL`, and the device plugin considers the GPU free once

the container is gone. A process that is still exiting can hold the GPU briefly and cause the next Pod on that GPU to fail. Handling SIGTERM correctly is therefore required for reliable GPU scheduling, not only good practice.

4.8 Node heartbeats and failure

Each node has a Lease object in the kube-node-lease namespace, which the kubelet renews regularly. The node lifecycle controller in kube-controller-manager:

- Checks node heartbeats every `--node-monitor-period` (default 5 seconds).
- Sets the node's Ready condition to Unknown when no heartbeat has arrived within `--node-monitor-grace-period` (default 40 seconds), and adds the node.kubernetes.io/unreachable or not-ready taint with the NoExecute effect.
- Lets Pods remain for their toleration period. By default, Pods receive a toleration of 300 seconds for these taints, so they are evicted about five minutes after the node becomes unreachable.
- Limits the rate of evictions with `--node-eviction-rate` (default 0.1 nodes per second, one node every 10 seconds), and reduces the rate further when more than `--unhealthy-zone-threshold` (default 55%) of a zone's nodes are unhealthy.

If all zones are unhealthy, the controller assumes that the control plane has lost connectivity rather than that every node has failed, and it stops evicting. A control plane network problem therefore cannot cause the whole cluster to be evicted.

For planned shutdowns, GracefulNodeShutdown (beta since v1.21) lets the kubelet detect the operating system shutdown and terminate Pods within a configured grace period.

4.9 Node-pressure eviction

Node-pressure eviction is performed by the kubelet, not through the Eviction API. The kubelet monitors eviction signals and evicts Pods when a threshold is crossed:

- **Signals:** `memory.available`, `nodefs.available`, `nodefs.inodesFree`, `imagefs.available`, `imagefs.inodesFree`, `containerfs.available`, `containerfs.inodesFree`, and `pid.available`.
- **Hard thresholds** (`evictionHard`) cause immediate eviction. **Soft thresholds** (`evictionSoft`) cause eviction only after the signal has stayed past the threshold for `evictionSoftGracePeriod`.
- **Defaults** on Linux include `memory.available<100Mi`, `nodefs.available<10%`, `imagefs.available<15%`, and `nodefs.inodesFree<5%`.
- **Order.** The kubelet first considers Pods whose usage exceeds their requests, ordered by priority and then by how far usage exceeds requests. BestEffort Pods have no requests, so they are evicted first. A Pod that stays within its requests is evicted only after all such Pods, so accurate requests protect a workload.

The scheduler placed the Pods, but it plays no part in node-pressure eviction. The kubelet acts on the node's actual resource use.

5. Controllers and informers

5.1 The controller pattern

A controller watches one or more resource types, compares the desired state in each object's spec with the observed state, and makes changes through the API until they match. A well-written controller is:

- **Level-triggered.** It reconciles the whole object rather than reacting to one event. A missed event does not matter, because the next reconcile reads the current state.
- **Idempotent.** Reconciling the same object twice with the same state produces the same result.
- **Self-healing.** If someone deletes a child object, the next reconcile recreates it.

Each reconcile reads the current state, computes the necessary changes, writes them, and returns. A controller that keeps its own record of what it did previously, and assumes it is still true, drifts from the actual state of the cluster.

5.2 client-go internals

controller-runtime and most operators are built on client-go, which provides these layers:

Layer	Role
Reflector	Lists a resource type and then watches it from the returned <code>resourceVersion</code> . On 410 Gone, it lists again. It is the only layer that reads from the API
DeltaFIFO	A queue of changes produced from the watch events
Indexer	A thread-safe local cache of objects, with indexes such as by namespace or by owner
Informer (SharedInformer)	Updates the cache from the queue and delivers add, update, and delete notifications. Several controllers can share one informer, and therefore one watch and one cache
Work queue	Holds object keys to reconcile. It deduplicates keys, retries failures with exponential backoff, and supports delayed requeues

If ten updates to one Deployment arrive while it is being reconciled, the work queue holds a single entry for it afterward. Controllers must therefore not depend on receiving every event. They must also expect the cache to be slightly out of date, and expect `resourceVersion` conflicts on writes.

5.3 controller-runtime terms

- Manager owns the clients, the shared cache, and leader election.
- `Reconciler.Reconcile(ctx, req)` receives a namespace and name, reads the object, and returns either a result, which can request a requeue after a delay, or an error.

- **Predicates** filter events, for example to reconcile only when the spec generation changes, so that the controller's own status updates do not trigger another reconcile.
- `client.Status().Update()` writes the status subresource, and `client.Update()` writes the spec and metadata. Using the wrong one is a common bug: the change is silently ignored.
- `Owns()` **and** `Watches()` watch child or related objects, so that a change to a child triggers a reconcile of its owner.
- **Finalizers** are added and removed explicitly, and a finalizer is removed only after its cleanup succeeds.

5.4 Avoiding conflicting writes

Three mechanisms prevent controllers from overwriting each other's changes:

1. **Optimistic concurrency.** Every update includes a `resourceVersion`, and an update based on an old version fails with `409 Conflict`.
2. **The status subresource.** The spec and the status are written through separate endpoints, so a controller that writes status cannot overwrite a user's spec change.
3. **Server-side apply.** `managedFields` records which manager owns each field. If two managers try to set the same field, apply reports a conflict.

In practice, an operator should read the latest object, change only the fields it manages, and handle conflicts by reading again rather than by retrying the same request. To take over a field that another manager owns, use server-side apply with forced ownership rather than repeatedly overwriting the other manager's value.

`04-kubernetes-operator.md` covers the `Reconcile` function, `CreateOrUpdate`, status conditions, and the controller-runtime builder in detail.

6. Services and cluster networking

6.1 Service types

Type	Behavior
ClusterIP (default)	A stable virtual IP address reachable inside the cluster
NodePort	Opens the same port on every node and forwards it to the Service
LoadBalancer	Asks the cloud provider, through the service controller, for an external load balancer
ExternalName	Returns a DNS CNAME record for an external name; no traffic is proxied
Headless (clusterIP: None)	No virtual IP; DNS returns the IP addresses of the Pods

The API server allocates cluster IPs from the Service CIDR, a range separate from the Pod CIDR, and releases them when the Service is deleted. A LoadBalancer Service receives the `service.kubernetes.io/load-balancer-cleanup` finalizer, so it remains in Terminating until the cloud load balancer has been deleted. A headless Service named in a StatefulSet's `serviceName` gives each Pod a DNS name such as `web-0.web.default.svc.cluster.local`.

The `externalIPs` field is deprecated as of v1.36. Use a load balancer or the Gateway API instead.

6.2 EndpointSlices

The EndpointSlice controller in `kube-controller-manager` creates EndpointSlice objects that list the Pods matching each Service's selector:

- **Slicing.** Endpoints are grouped by Service, IP family, protocol, and port, with up to `--max-endpoints-per-slice` endpoints in each slice (default 100, maximum 1000). Small slices keep updates small in large Services.
- **Conditions.** Each endpoint has three conditions. `serving` reflects the Pod's readiness. `terminating` is set when the Pod has a deletion timestamp. `ready` is `serving` and not `terminating`.
- **The legacy API.** `kube-proxy` uses EndpointSlices, not the older Endpoints API, which is deprecated as of v1.33. EndpointSlice mirroring from Endpoints objects is also deprecated; for a Service without a selector, create EndpointSlices directly.
- **Duplicates.** While updates propagate, the same endpoint can appear in more than one slice, so consumers must merge and deduplicate. `kube-proxy`'s `EndpointSliceCache` is the reference implementation.

During termination, a Pod's endpoint becomes `terminating: true` and `ready: false`, while `serving` still shows whether the Pod responds. This interval is when load balancers can drain connections.

6.3 How Service traffic is forwarded

kube-proxy programs forwarding on each node in one of three modes:

- `iptables` (the default in v1.37). kube-proxy installs iptables rules for each Service and endpoint. A connection to a cluster IP is translated with DNAT to one Pod's IP and port, and conntrack reverses the translation for replies. Rule processing cost grows with the number of rules, so very large clusters see slower updates, although partial syncs and `minSyncPeriod` reduce the effect.
- `ipvs`. kube-proxy uses the kernel's IPVS load balancer, which offers more balancing algorithms. The IPVS model fits the Service API poorly, and the mode is being removed: it is deprecated as of v1.35, the `KubeProxyIPVS` feature gate is deprecated in v1.37, support is disabled by default from v1.40, and it will be removed in v1.43. Do not choose it for a new cluster.
- `nftables` (stable in v1.33). An implementation based on nftables that performs better than iptables at scale. Upstream plans to make it the default in a future release; a v1.37 cluster uses iptables unless configured otherwise.

Other Service fields that affect routing:

- `sessionAffinity`: ClientIP sends connections from the same client IP to the same backend for a configurable time.
- `internalTrafficPolicy` and `externalTrafficPolicy` set to `Local` send traffic only to endpoints on the same node. Cluster uses all endpoints.
- `trafficDistribution` accepts `PreferSameZone` and `PreferSameNode` (`PreferClose` is a deprecated alias for `PreferSameZone`). It expresses a preference, and falls back to other endpoints, unlike the strict `Local` traffic policies.
- `ipFamilyPolicy` and `ipFamilies` configure dual-stack Services.

Cilium can replace kube-proxy and implement Services and network policy with eBPF programs in the kernel. It scales better, but the forwarding state can no longer be inspected with `iptables -L`, and operating it requires eBPF-specific tools.

6.4 DNS

CoreDNS provides cluster DNS. Each Pod's `/etc/resolv.conf` points to the cluster DNS Service and includes search domains for its namespace, so a Pod can resolve a Service by its short name.

- `ndots:5`. A name with fewer than five dots is first tried with each search domain appended, such as `api.example.com.default.svc.cluster.local`. Lookups of external names therefore generate several extra queries. Workloads with high query rates use fully qualified names with a trailing dot or lower `ndots` in `dnsConfig`.

- `dnsPolicy`. `ClusterFirst` (default) uses cluster DNS. `Default` uses the node's resolver. `ClusterFirstWithHostNet` is required for a Pod with `hostNetwork: true` to use cluster DNS. `None` requires the Pod to supply its own `dnsConfig`.

Any workload that resolves Service names depends on CoreDNS, and StatefulSets depend on it for stable Pod names.

6.5 CNI and Pod networking

The CNI plugin gives each Pod an IP address and connectivity. A typical plugin:

1. Creates a veth pair, with one end in the Pod's network namespace and one on the host.
2. Connects the host end to a bridge, Open vSwitch, or a routing or encapsulation device.
3. Assigns the Pod an IP address, usually from the node's Pod CIDR.
4. Adds routes and, if the plugin supports it, network policy rules.

CNI plugins differ in their routing model (overlay networks or native routing through the cloud network), IP address management, MTU, and whether they enforce policy with iptables or eBPF. An MTU that is too large for the underlying network causes connections that hang when large packets are sent. The problem is difficult to diagnose, and it also affects RDMA and GPUDirect configurations.

6.6 NetworkPolicy

A NetworkPolicy is a firewall for Pods, selected by labels:

- **Default allow.** A Pod that no policy selects accepts all traffic.
- **Selection enables isolation.** Once any policy selects a Pod for a direction, ingress or egress, only traffic that some policy allows in that direction is permitted.
- **Policies are additive.** The standard API has no deny rules; the allowed traffic is the union of all matching policies.
- **The CNI plugin enforces policies.** A plugin without policy support, such as a basic bridge plugin, ignores them silently. Calico and Cilium enforce them.

For a GPU cloud, start each tenant namespace with a default-deny policy and add explicit allow rules.

7. Storage, configuration, and secrets

7.1 PersistentVolumes, claims, and StorageClasses

Object	Scope	Role
PersistentVolumeClaim (PVC)	Namespace	A request for storage: size, access mode, and StorageClass
PersistentVolume (PV)	Cluster	A piece of storage. The control plane binds a PV to a PVC
StorageClass	Cluster	The provisioner (a CSI driver), its parameters, the reclaim policy, and the binding mode

Properties to know:

- **Access modes.** ReadWriteOnce allows one node to mount the volume for writing. ReadOnlyMany allows many nodes to read it. ReadWriteMany allows many nodes to write it, if the driver supports that. ReadWriteOncePod allows only one Pod.
- **Reclaim policy.** Delete removes the underlying storage when the PVC is deleted. Retain keeps it for manual cleanup. Recycle is deprecated.
- **Binding mode.** Immediate binds the PVC when it is created. WaitForFirstConsumer waits until a Pod using the PVC is scheduled. Use it for zonal storage, so the volume is created in the zone where the Pod runs.
- **Expansion.** A PVC can be enlarged if its StorageClass sets `allowVolumeExpansion: true` and the driver supports expansion.
- **Snapshots.** VolumeSnapshotClass and VolumeSnapshot create snapshots through CSI.
- **CSI.** The Container Storage Interface is the standard driver interface, and the former in-tree drivers have been migrated to CSI. A CSI driver typically runs as a DaemonSet on each node plus a controller Deployment for provisioning, attaching, and resizing.

GPU training usually reads datasets from high-throughput shared storage, such as Lustre, Weka, NFS, or a cloud parallel file system, and writes checkpoints to fast local storage.

7.2 ConfigMaps and Secrets

- **ConfigMaps** hold configuration that is not sensitive. Pods consume them as environment variables, files, or command-line arguments.
- **Secrets** hold sensitive data. Their values are **base64-encoded, not encrypted**. Encryption at rest with KMS v2 protects the data in etcd, RBAC controls who can read it through the API, and an external secret store such as Vault or a cloud secret manager keeps it out of the cluster.

- **Immutable objects.** Setting `immutable: true` stops the kubelet from watching the object for changes, which reduces API server load in large clusters. To change the data, create a new object.
- **Projected volumes** combine several sources, such as ConfigMaps, Secrets, the Downward API, and ServiceAccount tokens, into one directory.

ServiceAccount tokens are handled as described in section 1.3: short-lived, audience-bound, and rotated automatically.

7.3 ResourceQuota and LimitRange

- **ResourceQuota** limits total resource use in a namespace: compute resources, storage, and object counts. A quota can apply to a subset of Pods through scopes such as `BestEffort`, `NotBestEffort`, `Terminating`, `NotTerminating`, `PriorityClass`, and `CrossNamespacePodAffinity`. When a quota limits a resource, every new Pod must specify a request or limit for that resource.
- **LimitRange** sets default requests and limits, minimum and maximum values per container or Pod, and a maximum ratio of limit to request. It is enforced by admission when Pods are created, not by the scheduler.

For a GPU cloud, Kubernetes quota is necessary but not sufficient. A quota on `nvidia.com/gpu` counts devices without distinguishing a whole GPU from a MIG instance or a time-sliced replica, and it knows nothing about reservations or billing. The control plane needs its own reservation and billing system.

8. Scheduling GPU workloads

8.1 Extended resources

`nvidia.com/gpu` is an **extended resource**: a resource with a fully qualified name outside the `kubernetes.io` domain. Extended resources have specific rules:

- **Integers only.** They cannot be requested in fractions and cannot be overcommitted.
- **Limits are required.** An extended resource must appear in `limits`. If `requests` is also set, it must equal the limit; if it is omitted, it defaults to the limit.
- **No sharing through this path.** Each unit is allocated to one container.
- **Opaque to the scheduler.** The scheduler counts units on each node but does not know which physical GPU a unit represents. The kubelet and the device plugin choose the device on the node.

```
apiVersion: v1
kind: Pod
metadata:
  name: triton-server
spec:
  containers:
    - name: triton
      image: nvcr.io/nvidia/tritonserver:latest
      resources:
        limits:
          nvidia.com/gpu: 1      # integer; no requests value needed
```

Set CPU and memory requests for GPU Pods as well. QoS classes consider only CPU and memory, so a Pod that requests a GPU but no CPU or memory is `BestEffort`: the scheduler does not reserve CPU or memory for it, and it is among the first Pods evicted under node pressure.

8.2 How a device plugin works

A device plugin is a gRPC server on each node, usually deployed as a `DaemonSet`, that registers with the kubelet through a Unix socket:

```

Device plugin (DaemonSet, privileged, mounts /var/lib/kubelet/device-plugins)
| 1. Register(ResourceName=nvidia.com/gpu, socket, API version)
▼
kubelet Device Manager: ListAndWatch keeps device health up to date
| 2. Node status advertises capacity/allocatable: nvidia.com/gpu
▼
scheduler places a Pod requesting a GPU on a node with allocatable capacity
| 3. kubelet calls Allocate(deviceIDs) for the container
▼
AllocateResponse: device nodes, mounts, env vars, annotations, or CDI device names
|
▼
container runtime injects the GPU; container starts with CUDA available

```

The plugin implements these methods:

Method	Purpose
GetDevicePluginOptions	Reports which optional methods the plugin supports
ListAndWatch	Streams the list of devices and their health
Allocate	Returns the runtime settings for the devices assigned to a container
GetPreferredAllocation	Optional. Suggests which devices to assign together, such as GPUs connected by NVLink or on the same NUMA node
PreStartContainer	Optional. Prepares or resets devices before a container starts

Operational details:

- **Registration order.** The plugin must be serving its gRPC socket before it calls Register on /var/lib/kubelet/device-plugins/kubelet.sock, because the kubelet connects back immediately.
- **Kubelet restarts.** When the kubelet restarts, it deletes the sockets in the device plugin directory. The plugin must detect this and register again.
- **Device failures.** When a GPU fails, for example with an Xid error, uncorrectable ECC errors, or overheating, the plugin reports it as unhealthy through ListAndWatch. The kubelet reduces the node's *allocatable* count, not its capacity, so no new Pods are assigned the device. Pods already using it keep it and usually fail.
- **CDI device names.** Since v1.31 (DevicePluginCDIDevices, GA), a plugin can return fully qualified CDI device names in its Allocate response. The container runtime then adds the devices through CDI rather than through environment variables read by a runtime hook.

8.3 GPU resources and labels on a node

When a device plugin is registered and healthy, `kubectl describe node <gpu-node>` shows `nvidia.com/gpu` under Capacity and Allocatable. If it is missing, the plugin is not registered, the plugin reports no healthy devices, or the driver is not working.

The NVIDIA device plugin can advertise these resource names:

Resource name	Meaning
<code>nvidia.com/gpu</code>	Whole GPUs, or MIG instances with the single strategy
<code>nvidia.com/mig-<slice>g.<memory>gb</code>	MIG instances of a specific profile, with the mixed strategy
A renamed resource such as <code>nvidia.com/gpu.shared</code>	Time-sliced or MPS replicas, when the plugin is configured with <code>renameByDefault: true</code>

GPU Feature Discovery and Node Feature Discovery add node labels for placement, such as `nvidia.com/gpu.product`, `nvidia.com/gpu.memory`, `nvidia.com/gpu.count`, `nvidia.com/gpu.family`, `nvidia.com/gpu.replicas`, `nvidia.com/gpu.sharing-strategy`, `nvidia.com/mig.capable`, `nvidia.com/mig.strategy`, and CUDA driver and runtime versions. The exact set depends on the version and configuration, so list the labels on a node rather than relying on a fixed list:

```
kubectl get node <node> -o json \
  | jq -r '.metadata.labels | to_entries[]
    | select(.key | startswith("nvidia.com/")) | "\(.key)=\(.value)''
```

Select nodes with those labels in a node selector or affinity rule:

```
spec:
  nodeSelector:
    nvidia.com/gpu.product: NVIDIA-A100-SXM4-80GB
```

8.4 MIG, time-slicing, and MPS

GPUs can be shared in several ways, and each maps to Kubernetes differently:

- **MIG (Multi-Instance GPU)** divides a supported GPU into instances with dedicated memory and compute. With the device plugin's `migStrategy: single`, every MIG instance is advertised as `nvidia.com/gpu`. With `mixed`, whole GPUs are advertised as `nvidia.com/gpu` and each MIG profile has its own resource name, `nvidia.com/mig-<slice>g.<memory>gb`. MIG isolates memory and faults between instances, which multi-tenant inference requires.
- **Time-slicing** lets several containers share a GPU by taking turns. It provides no memory isolation and no fault isolation: one workload that exhausts GPU memory or crashes the GPU context can affect the others. The plugin configuration sets replicas, and the plugin advertises that many units for each GPU.
- **MPS (Multi-Process Service)** runs several processes in a shared GPU context, with configurable limits on memory and compute share. In the NVIDIA device plugin, time-slicing and MPS cannot be used together, and the sharing strategy applies to all GPUs on a node.

Use MIG when tenants must not affect each other. Use time-slicing or MPS for workloads where higher utilization matters more than isolation.

`10-gpu-operator-kubebuilder-deep-dive.md` describes how the GPU Operator configures these modes.

8.5 NUMA alignment

GPU performance depends on the placement of the GPU, the CPU cores, the memory, and the network card relative to each other. Several kubelet components align them:

- **The CPU Manager** with the static policy gives exclusive CPU cores to containers in Guaranteed Pods that request whole CPUs.
- **The Topology Manager** aligns CPU, memory, and device allocations to the same NUMA node, according to its policy: none, best-effort, restricted, or single-numa-node. Device plugins provide NUMA information in `TopologyInfo`; without it, the Topology Manager cannot align devices.
- **The Memory Manager** allocates memory and huge pages from specific NUMA nodes.

A low-latency inference node aims to give each workload exclusive cores, its GPU, and its network card on one NUMA node, with the GPUs connected appropriately over PCIe or NVLink. This requires node configuration, not only fields in the Pod spec.

8.6 Dynamic Resource Allocation (DRA)

DRA replaces integer counting with structured requests for devices. Its API objects are in `resource.k8s.io/v1`:

Object	Role
DeviceClass	A category of devices, with CEL selectors and an optional <code>extendedResourceName</code>
ResourceClaim	A request for devices. Several Pods can reference the same claim
ResourceClaimTemplate	A template from which a separate ResourceClaim is created for each Pod
ResourceSlice	A driver-published list of devices on a node, with their attributes and capacity

The allocation process:

1. A DRA driver publishes `ResourceSlice` objects that describe the devices on each node.
2. A user or controller creates a `ResourceClaimTemplate` for per-Pod devices, or a `ResourceClaim` for devices shared by several Pods.
3. The control plane creates a `ResourceClaim` for each Pod from its template.

4. The scheduler finds nodes whose `ResourceSlice` devices satisfy the claim and records the chosen devices in the claim's status.
5. The kubelet calls the driver's `NodePrepareResources`, and the driver prepares the devices and returns CDI device names.
6. When the Pod ends, the kubelet calls `NodeUnprepareResources`.

What DRA adds compared with device plugins:

- **Constraints written in CEL** on device attributes, for example selecting devices with more than a certain amount of memory, instead of defining a resource name for every device model.
- **Requests for attributes and capacity**, not only a count.
- **Claims with their own lifecycle**. A claim can outlive a Pod or be shared by several Pods.
- **Partitionable devices** (`DRAPartitionableDevices`, beta in v1.36), which let one physical device be offered as several devices, as MIG does.
- **Extended resources backed by DRA** (`DRAExtendedResource`: alpha in v1.34, beta in v1.36, stable in v1.37). A `DeviceClass` can declare an `extendedResourceName`, so existing Pods that request, for example, `example.com/gpu: 2` continue to work while DRA performs the allocation. The special resource name `deviceclass.resource.kubernetes.io/<class>` allocates from a device class without a separate claim.
- **Prioritized alternatives** (`DRAPrioritizedList`, stable in v1.36), so a claim can list acceptable device types in order of preference.
- **Monitoring integration**. The kubelet's `PodResources` API reports DRA allocations, so monitoring agents can attribute a device to a Pod.

	Device plugin	DRA
Allocation model	Integer extended resource	Structured parameters matched against <code>ResourceSlice</code> devices
Constraints	Resource names and node labels	CEL expressions over device attributes and capacity
Lifecycle	Tied to the Pod	Independent claims that can be shared or created from templates
Maturity	Stable and widely deployed	Core API stable since v1.34; more features still arriving
Suited to	Whole GPUs, and MIG through named resources	Mixed device types, dynamic partitioning, and complex placement

In current production deployments, the GPU Operator still uses the device plugin, and NVIDIA is developing DRA support in parallel.

8.7 Gang scheduling

Distributed training needs **all-or-nothing** placement. If a job needs eight workers and only seven can be placed, the seven that start hold their GPUs while waiting for the eighth, wasting capacity, and two partially placed jobs can block each other indefinitely.

The GangScheduling scheduler plugin (alpha in v1.35) implements all-or-nothing placement. In v1.37 it is part of the GenericWorkload feature gate (alpha in v1.35, beta in v1.37, disabled by default), together with the Workload and PodGroup APIs and workload-aware preemption. As documented upstream, it works as follows:

1. The scheduler holds Pods at PreEnqueue until their PodGroup exists and the number of Pods in it reaches the policy's minCount.
2. The scheduler then evaluates the group's placement in a single scheduling cycle, and a PlacementFeasible check tracks whether minCount can still be satisfied.
3. If at least minCount Pods can be placed, they are bound. Otherwise, none are bound, and the resources remain available for other workloads.

CompositePodGroup (alpha in v1.37) adds groups of groups with a minGroupCount, for training jobs made of several replicated components. On clusters without these features, a custom scheduler plugin can implement all-or-nothing placement at the Permit extension point, which is how projects such as Volcano and the scheduler-plugins coscheduling plugin have done it.

8.8 Requirements for a working GPU Pod

A GPU Pod runs successfully only when all of the following are in place, in this order:

1. A supported GPU driver on the host, installed directly or by a driver container.
2. The NVIDIA Container Toolkit, so the runtime can add driver libraries and device nodes to containers.
3. A healthy device plugin advertising nvidia.com/gpu, or a DRA driver publishing ResourceSlice objects.
4. Node labels that identify the GPU model and configuration.
5. A Pod that requests the GPU in limits, with CPU and memory requests.
6. A container image whose CUDA version the host driver supports.

The GPU Operator automates steps 1 through 4; see `10-gpu-operator-kubebuilder-deep-dive.md`.

9. Multi-tenancy and GPU cloud control planes

9.1 Soft and hard multi-tenancy

The Kubernetes documentation does not define a single model of multi-tenancy. A useful distinction is:

- **Soft multi-tenancy.** Tenants trust each other, such as teams in one company. Namespaces, RBAC, quotas, and NetworkPolicy are usually sufficient.
- **Hard multi-tenancy.** Tenants may be hostile, as in a public GPU cloud. The boundary must be stronger: dedicated nodes, sandboxed runtimes such as gVisor or Kata Containers, separate clusters, or virtual machines.

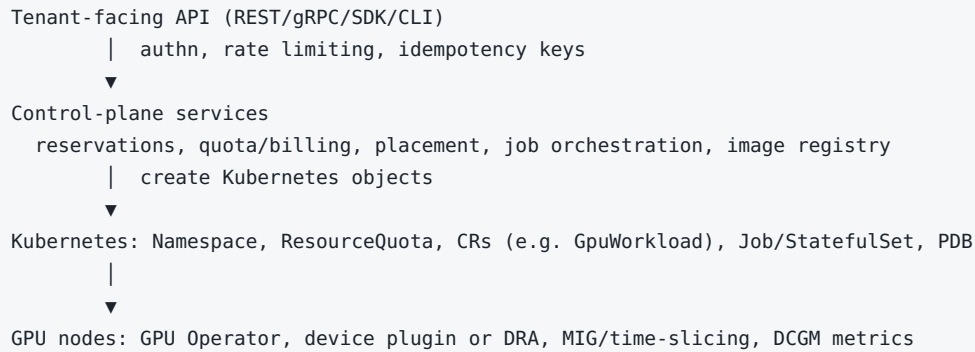
A namespace is not a security boundary. A privileged Pod, a hostPath volume, or a container escape crosses it. Pod Security Admission with the baseline or restricted profile prevents the most dangerous Pod configurations, but a hard boundary requires isolation at the kernel or hypervisor level.

9.2 Controls by layer

Layer	Controls
Identity	OIDC for users, projected ServiceAccount tokens for workloads, least-privilege RBAC, no shared cluster-admin
Admission	Pod Security Admission, LimitRange, ResourceQuota, ValidatingAdmissionPolicy, and tenant-aware webhooks
Network	A default-deny NetworkPolicy in each namespace, egress policies, and separate networks for tenant traffic where needed
Compute	Quotas, priority classes, taints, dedicated node pools, and MIG for GPU isolation
Runtime	seccomp, AppArmor or SELinux, read-only root filesystems, no privileged containers, and sandboxed runtime classes
Storage	StorageClasses and CSI credentials for each tenant; no hostPath volumes
Control plane	API Priority and Fairness to limit one tenant's API load; audit logging
Workload fairness	QoS classes, NUMA alignment, and a GPU sharing policy (MIG or time-slicing)

9.3 A GPU cloud control plane built on Kubernetes

A request such as "four A100 GPUs for eight hours" does not map to a single Kubernetes object. A typical design has several layers:



Design considerations:

- **Quota has two layers.** Kubernetes ResourceQuota limits each cluster, but the product also needs reservations and billing outside Kubernetes, kept consistent with the clusters.
- **Placement has several objectives.** Matching a GPU model or MIG profile, respecting topology, sharing capacity fairly among tenants, and minimizing cost can conflict, and the placement system has to choose which to prioritize.
- **Node failures are routine.** A workload with a reservation must be rescheduled when its node fails, and its reservation must not be lost or counted twice.
- **The API must be idempotent.** Clients retry, so a create request needs an idempotency key and a reconciliation process.
- **Observability is part of the product.** DCGM metrics, usage attributed to tenants, and GPU health data such as Xid and ECC errors are required for billing and for meeting service-level agreements.

06-system-design-cloud-gpu.md develops this design as a system design interview answer.

10. kubectl, client-go, and apply

10.1 kubectl commands

```
# Cluster and node state
kubectl get nodes -o wide
kubectl describe node <node> # capacity, allocatable, taints, events
kubectl get nodes -o custom-columns=\
NAME:.metadata.name,GPU:.status.allocatable.nvidia\\.com/gpu

# Workload debugging
kubectl get pods -A -o wide --field-selector=status.phase!=Running
kubectl describe pod <pod> # events, conditions, container states
kubectl logs <pod> --previous --tail=200
kubectl logs <pod> -c <container> -f
kubectl exec -it <pod> -- sh
kubectl debug -it <pod> --image=busybox --target=<container> # ephemeral container

# Events and rollout
kubectl get events -A --sort-by=.lastTimestamp
kubectl rollout status deployment/<name>
kubectl rollout undo deployment/<name>
kubectl rollout restart deployment/<name>

# API surface and validation
kubectl api-resources | grep -i gpu
kubectl explain pod.spec.containers --recursive
kubectl apply --dry-run=server -f manifest.yaml
kubectl diff -f manifest.yaml

# JSONPath, useful in scripts
kubectl get pod <pod> -o jsonpath='{.status.phase}'
kubectl get pvc -o jsonpath='{range .items[*]}{.metadata.name}{\t}{.status.phase}{\n}{end}'
```

`kubectl debug` adds an ephemeral container (`EphemeralContainers`, stable in v1.25) to a running Pod without restarting it. Use it to get a shell and debugging tools in a Pod whose image has no shell, such as a distroless image.

10.2 Calling the API directly

`kubectl` is an authenticated HTTP client. You can make the same requests with `curl`:

```
APISERVER=$(kubectl config view --minify -o jsonpath='{.clusters[0].cluster.server}')
TOKEN=$(kubectl create token default)
curl -sS -H "Authorization: Bearer $TOKEN" "$APISERVER/api/v1/namespaces/default/pods" | jq
'.items[] .metadata.name'
```

Every Kubernetes client library sends requests like these. Making one by hand shows how bearer tokens and API group paths work. Depending on the cluster, `curl` may also need the cluster's CA certificate (`--cacert`), and the default `ServiceAccount` needs RBAC permission to list Pods.

10.3 client-go and controller-runtime

- **client-go** provides typed clients, informers, listers, and work queues. It is the foundation of Go tooling for Kubernetes.
- **controller-runtime**, used by Kubebuilder and the Operator SDK, wraps the informers, cache, and work queue in a Manager and a Reconciler, and adds scheme registration, watch configuration, leader election, metrics, and webhook support.

Use controller-runtime for an operator. A CLI or a simple client needs only a client-go clientset.

10.4 Create, replace, apply, and patch

Command	Behavior
kubectl create	Fails if the object already exists
kubectl replace	Replaces the whole object. Fields that you omit are removed
kubectl apply	Sends a patch. Client-side apply compares with the last-applied-configuration annotation; server-side apply (<code>--server-side</code>) uses managedFields and field ownership
kubectl patch	Applies a strategic merge patch, JSON merge patch, or JSON patch. Add <code>--subresource=status</code> to patch the status

For an operator that owns only part of an object, use server-side apply or a targeted patch rather than reading, modifying, and updating the whole object. When a read-modify-update cycle is necessary, handle 409 Conflict by reading the object again and recomputing the change, not by resending the same request.

11. Debugging scenarios

Work through the layers in order: the API object, the scheduler's decision, the kubelet and container runtime, the node, and finally the GPU. There is no reason to run `nvidia-smi` for a Pod that has not been scheduled.

11.1 A Pod stuck in Pending

A Pending Pod is not running. It may not be scheduled, or it may be scheduled with its containers not yet created. The Pod's events contain the scheduler's reason.

```
kubectl describe pod <pod>           # read Events and Conditions
kubectl get events --field-selector involvedObject.name=<pod>
kubectl get nodes -o wide
kubectl describe node <node> | sed -n '/Allocatable/,/System Info/p'
kubectl get pvc
kubectl get resourcequota,limitrange -n <ns>
```

Cause	Typical message or signal
Not enough resources	0/3 nodes are available: 3 Insufficient nvidia.com/gpu
Taints or affinity	node(s) had untolerated taint, or node(s) didn't match Pod's node affinity/selector
Volumes	pod has unbound immediate PersistentVolumeClaims, or a volume zone conflict
Quota or LimitRange	The API server rejects the Pod, so no Pod exists. Look for a FailedCreate event on the owning ReplicaSet or Job
Scheduling gates	spec.schedulingGates (stable in v1.30) keeps the Pod out of the scheduling queue until a controller removes the gate

11.2 A Pod stuck in ContainerCreating

The Pod is scheduled, and the kubelet on its node is responsible for the next steps:

```
kubectl describe pod <pod>           # image pull, volume mount, sandbox events
journalctl -u kubelet -n 200 --no-pager
crictl ps -a
crictl logs <container-id>
journalctl -u containerd -n 200 --no-pager
```

Common causes are image pull failures (credentials or network), a CSI volume that never attaches or mounts, a CNI failure creating the Pod sandbox, and a device plugin or DRA driver failure. On GPU nodes, a FailedCreatePodSandBox event with a driver or runtime error usually indicates a problem with the runtime hook or the CDI configuration, not with scheduling.

11.3 A container in CrashLoopBackOff

```
kubectl logs <pod> --previous
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[*].lastState}'
kubectl describe pod <pod>
```

Start with the exit code of the last terminated container:

- **137** means the process received SIGKILL. If `lastState.terminated.reason` is `OOMKilled`, the memory limit is too low or the process leaks memory; the `oom_kill` counter in the container's cgroup memory.events file confirms it. Otherwise, check for a failing liveness probe.
- **143** means the process received SIGTERM.
- **Other codes** come from the application; read its logs.

CrashLoopBackOff describes the kubelet's restart delay. It is a symptom, not a diagnosis.

11.4 A node in NotReady

```
kubectl get node <node> -o yaml | grep -A20 conditions
kubectl describe node <node> | sed -n '/Conditions:/,/Addresses:/p'
systemctl status kubelet containerd
journalctl -u kubelet -n 300 --no-pager
df -h; free -h; uptime
dmesg -T | tail -50
```

Possible causes include a stopped kubelet, a failed container runtime, disk or inode exhaustion, process ID exhaustion, a network partition between the node and the control plane, and kernel problems. The node controller sets Ready to Unknown whenever heartbeats stop, including when the kubelet is running but cannot reach the API server or is starved of CPU.

11.5 A Service with endpoints that does not respond

```
kubectl get endpointslice -l kubernetes.io/service-name=<svc> -o yaml
kubectl get svc <svc> -o yaml
kubectl get pod -o wide -l <selector>
kubectl exec -it <client> -- wget -q0- <podIP>:<targetPort> # bypass the Service
```

Check, in order:

1. The EndpointSlices contain at least one endpoint with `ready: true`.
2. The Service's `targetPort` matches the port the container listens on.
3. No NetworkPolicy blocks traffic from the client.

If connecting to the Pod IP works but connecting to the Service IP does not, the problem is in kube-proxy, the CNI plugin's forwarding, or conntrack, not in the application. If only some requests fail during a rollout, look for traffic still reaching terminating endpoints.

11.6 DNS failures

```
kubectl exec -it <pod> -- cat /etc/resolv.conf
kubectl exec -it <pod> -- nslookup kubernetes.default.svc.cluster.local
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
```

Typical causes are CoreDNS Pods that are down or overloaded, excessive lookups caused by ndots, a NetworkPolicy that blocks UDP or TCP port 53, and a Pod with `hostNetwork: true` that lacks `dnsPolicy: ClusterFirstWithHostNet`.

11.7 A GPU Pod cannot use the GPU

The checks follow the requirements in section 8.8:

```
# 1. Host sees the GPU?
nvidia-smi

# 2. Device plugin healthy and registered?
kubectl get pods -n gpu-operator -l app=nvidia-device-plugin-daemonset
kubectl logs -n gpu-operator -l app=nvidia-device-plugin-daemonset --tail=100

# 3. Node advertises the resource?
kubectl describe node <node> | grep -A6 'nvidia.com/gpu'

# 4. Pod request correct? (limits only; integer)
kubectl get pod <pod> -o jsonpath='{.spec.containers[*].resources}'

# 5. Runtime injection works?
kubectl exec <pod> -- nvidia-smi

# 6. Hardware errors?
dmesg -T | grep -i xid
nvidia-smi -q -d ECC,PAGE_RETIREMENT
```

Interpret the results:

- The node advertises GPUs, but the Pod is Pending: the problem is capacity or node selection.
- The Pod runs, but `nvidia-smi` is missing or finds no devices: the container toolkit did not add the GPU to the container.
- `nvidia-smi` works, but the workload fails: check CUDA and driver compatibility, and look for Xid errors.

11.8 Controller and API server problems

```
kubectl -n <ns> logs deploy/<operator> --tail=200
kubectl get <crd-object> -o yaml | grep -A5 status
kubectl get lease -A
kubectl get --raw /metrics | grep apiserver_request_total | head
```

Symptom	Likely cause and fix
Objects stuck in Terminating	A finalizer that no running controller removes
A controller reconciling constantly	Its own status writes trigger reconciles; add a predicate or write status only when it changes
Many 409 Conflict errors	Several writers modify the same fields; use server-side apply and field ownership
High API latency and 429 responses	API Priority and Fairness throttling, or etcd under load
Leader election changing frequently	etcd latency, or network problems between the controller and the API server

12. Interview questions

What happens between `kubectl apply` and a running Pod?

The API server authenticates the request, authorizes it with RBAC, runs mutating admission (defaults and sidecar injection), validates the object against its schema, runs validating admission, converts it to the storage version, and writes it to etcd with optimistic concurrency. Watchers are notified. The Deployment controller creates a ReplicaSet with an owner reference, and the ReplicaSet controller creates Pods. The scheduler filters and scores nodes, reserves resources, and writes `spec.nodeName` through the Binding subresource. The kubelet on that node admits the Pod, mounts volumes, asks the runtime to create a sandbox, configures networking through CNI, starts init containers, sidecars, and application containers, and updates the Pod's status. The EndpointSlice controller adds the Pod to the Service's endpoints when it is ready. Every step after the write to etcd is asynchronous.

Follow-up: what if the Deployment controller is not running when you apply? The Deployment is stored. When the controller starts, it lists all Deployments and reconciles each one, because controllers are level-triggered.

What happens when a node fails while running Pods?

The kubelet stops renewing its Lease. After the grace period, the node lifecycle controller sets the node's Ready condition to Unknown and adds the `node.kubernetes.io/unreachable` taint with the NoExecute effect. Pods are evicted when their toleration for the taint expires, 300 seconds by default, at a rate limited per zone. The owning controllers create replacement Pods on other nodes if capacity is available. The old Pod objects can remain visible until eviction completes, so seeing Pods on a node that no longer exists is expected for a few minutes.

Why is a Deployment rollout stuck?

The rollout waits for Pods in the new ReplicaSet to become ready. Common reasons are image pull failures, failing readiness probes, requests that do not fit on any node, taints, unbound PVCs, quota, and a `maxUnavailable` setting that allows no progress. Use `kubectl rollout status` and the events to find the stage. After `progressDeadlineSeconds`, the Deployment is marked as failed, but it is not rolled back automatically.

How does a Service route only to healthy Pods?

The EndpointSlice controller lists the Pods that match the Service's selector and sets each endpoint's conditions from the Pod's readiness and termination state. kube-proxy, or a replacement such as Cilium, forwards traffic only to ready endpoints. A Pod that fails its readiness probe is removed from the endpoints without being restarted.

How are GPU requests scheduled?

A healthy device plugin makes the node advertise an integer `nvidia.com/gpu` allocatable count. The scheduler treats the count like any other integer resource that cannot be overcommitted and places the Pod on a node with enough units. The kubelet then calls the plugin's `Allocate` method, which maps the units to specific devices and returns the device nodes, mounts, or CDI names. With DRA, the scheduler instead matches `ResourceSlice` devices against the claim's structured parameters and CEL constraints.

Follow-up: where does the device plugin run, and what chooses the device? It runs as a `DaemonSet` and registers with the kubelet over a Unix socket. The kubelet's device manager chooses the devices, optionally guided by `GetPreferredAllocation` and NUMA topology, and calls `Allocate`.

Should MIG be exposed through the device plugin or DRA?

The device plugin with `migStrategy: mixed` is the established approach and advertises a resource name for each MIG profile. For new designs that need richer constraints, dynamic partitioning, or claims that outlive Pods, DRA is the direction upstream is taking: partitionable devices (beta in v1.36) model MIG-style partitioning directly, and DRA-backed extended resources (stable in v1.37) keep existing `nvidia.com/gpu` requests working. The choice depends on the drivers available and the cluster's version.

Why does distributed training need gang scheduling?

If only some workers are placed, they hold GPUs while waiting for the rest, and two partially placed jobs can wait on each other indefinitely. Gang scheduling places all required Pods or none. The `Workload` and `PodGroup` APIs (beta in v1.37, behind the `GenericWorkload` feature gate) express the minimum group size with `minCount` and `minGroupCount`; custom schedulers implement the same behavior at the `Permit` extension point.

What is the difference between requests and limits?

Requests determine scheduling and QoS class; limits are enforced by the kernel. A container that exceeds its memory limit is OOM-killed, and one that exceeds its CPU limit is throttled. Requests also affect node-pressure eviction: Pods that use more than their requests are evicted before Pods that stay within them.

What happens if etcd is slow or lost?

`etcd` stores all cluster state, is replicated with Raft, and is written only by the API server. When it is slow, writes are slow, watch notifications are delayed, and controllers repeatedly lose leader election. Running containers continue, because the kubelet and container runtime do not need `etcd` to keep them running, but nothing new can be scheduled or changed. If `etcd` is lost without a backup, the cluster state is lost.

How would you isolate two teams on one GPU cluster?

For teams that trust each other: a namespace per team, least-privilege RBAC, ResourceQuota and LimitRange, Pod Security Admission with the restricted profile, a default-deny NetworkPolicy, separate ServiceAccounts, and audit logging. For stronger isolation, add dedicated node pools with taints and tolerations, MIG for GPU memory and fault isolation, sandboxed runtimes or separate clusters, and tenant-aware admission policies. A namespace alone is not a security boundary.

A controller is reconciling constantly and the API server is slow. What do you check?

Check whether the controller's own status writes trigger new reconciles, whether a broad watch fills the work queue, whether the controller writes on every reconcile instead of only when something changed, and whether it lists from the API instead of using its informer cache. API Priority and Fairness throttling appears as 429 responses, and the work queue metrics show the controller's backlog. Predicates, comparing before writing, and rate limiting address these causes.

Summary

- The API server is the only writer to etcd. Every request passes through authentication, authorization, mutating admission, validation, and validating admission before it is stored.
- Controllers are level-triggered and idempotent. client-go informers cache objects, and work queues deduplicate and retry work.
- The scheduler only binds Pods to nodes. The kubelet runs Pods, reports status, and evicts Pods under node pressure.
- Requests drive scheduling, QoS, and eviction order; limits are enforced by the kernel.
- EndpointSlices, kube-proxy, CoreDNS, and the CNI plugin together make Services reachable, and NetworkPolicy is enforced only by CNI plugins that support it.
- GPUs are extended resources advertised by device plugins. DRA adds structured, attribute-based allocation, and gang scheduling provides all-or-nothing placement.
- Namespaces are not a security boundary; hard multi-tenancy needs node, runtime, or cluster-level isolation.
- Debug from the API object down through the scheduler, kubelet, node, and GPU.

Further reading

The upstream documentation is the authoritative source.

- Kubernetes documentation, concepts: <https://kubernetes.io/docs/concepts/>
- Cluster architecture: <https://kubernetes.io/docs/concepts/architecture/>
- Controllers: <https://kubernetes.io/docs/concepts/architecture/controller/>
- Garbage collection: <https://kubernetes.io/docs/concepts/architecture/garbage-collection/>
- Nodes and heartbeats: <https://kubernetes.io/docs/concepts/architecture/nodes/>
- Pod lifecycle: <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
- Sidecar containers: <https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/>
- Pod QoS: <https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>
- Scheduling framework: <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>
- Node-pressure eviction: <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>
- Services and virtual IPs: <https://kubernetes.io/docs/reference/networking/virtual-ips/>
- EndpointSlices: <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>
- Device plugins: <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/>
- DRA: <https://kubernetes.io/docs/concepts/resource-management/dynamic-resource-allocation/>
- Gang scheduling: <https://kubernetes.io/docs/concepts/scheduling-eviction/gang-scheduling/>
- Multi-tenancy: <https://kubernetes.io/docs/concepts/security/multi-tenancy/>
- API Priority and Fairness: <https://kubernetes.io/docs/concepts/cluster-administration/flow-control/>
- Feature gates (version history): <https://kubernetes.io/docs/reference/command-line-tools-reference/feature-gates/>
- Kubernetes Enhancement Proposals: <https://github.com/kubernetes/enhancements>
- KEP-624: Scheduling framework
- KEP-753: Sidecar containers
- KEP-4381: Dynamic Resource Allocation with structured parameters
- Books:
 - Brendan Burns, Joe Beda, Kelsey Hightower, and Lachlan Evenson, *Kubernetes: Up and Running*
 - Marko Lukša, *Kubernetes in Action*
 - Michael Hausenblas and Stefan Schimanski, *Programming Kubernetes*, on client-go and operators
 - Brendan Burns, *Designing Distributed Systems*, on controller and sidecar patterns

- NVIDIA projects:
- Device plugin: <https://github.com/NVIDIA/k8s-device-plugin>
- GPU Operator: <https://github.com/NVIDIA/gpu-operator>
- GPU Feature Discovery: <https://github.com/NVIDIA/gpu-feature-discovery>

Source: [03-kubernetes-concepts.md](#) · [PDF](#) · [All study guides](#)